

Answer Key for EDB Postgres AI Database Advanced v17





Table of Contents

Database Monitoring	4
SQL Tuning	9
Performance Tuning	12
Extensions	14
Table Partitioning	15

Database Monitoring

Lab Exercise 1

You have been asked to install and configure Postgres Enterprise Manager and monitor a Postgres instance running on a Linux machine

1. Prepare a Linux machine for PEM Server installation
2. Verify backend database
3. Install and configure PEM server
4. Open PEM Web Interface and login to PEM Server

Optional

1. Install PEM Agent on the managed host
2. Register PEM Agent with PEM Server
3. Verify PEM Agent information in PEM Web Interface

Answer Lab Exercise – 1

1. To install **Postgres Enterprise Manager Server** on Rocky Linux 9 we need to use packages from <https://www.enterprisedb.com/repos-downloads>

- i. Prepare a Linux machine for PEM installation:

- a. Login as sudo or root user and configure EDB Postgres[®] AI repositories:

```
[root@edblab ~]# curl -sLf 'https://downloads.enterprisedb.com/<token>/enterprise/setup.rpm.sh' | sudo -E bash
```

- ii. Verify backend database for PEM (Enterprise Postgres or PostgreSQL 12 or higher):

- a. Here we have Enterprise Postgres(Oracle Compatible) 17 already installed and the default database cluster is up and running on port 5444
- b. Verify Connection as below and exit psql

```
[root@localhost ~]# su - enterprisedb
[enterprisedb@localhost ~]$ /usr/edb/as17/bin/psql edb
Password for user enterprisedb:
psql (17.2.0)
Type "help" for help.
```

```
edb=# \q
[enterprisedb@localhost ~]$ █
```

- iii. To install PEM Server:

- a. Install PEM using dnf command

```
# dnf config-manager --set-enabled crb
# dnf -y install edb-pem
```

- b. Run PEM configuration file with following command and provide the details as highlighted below:

```
[root@localhost ~]# /usr/edb/pem/bin/configure-  
pem-server.sh
```

```
-----  
EDB Postgres Enterprise Manager  
-----
```

```
Install type: 1:Web Services and Database, 2:Web  
Services 3: Database [ ] :1
```

```
##We are installing Web and Database services  
locally.
```

```
Enter local database server installation path  
(i.e. /usr/edb/as12 , or /usr/pgsql-12, etc.) [ ]  
:/usr/edb/as17
```

```
##Provide Path to EDB installation
```

```
Enter database super user name [ ] :enterprisedb  
##Enter database superuser name
```

```
Enter database server port number [ ] :5444  
##Enter the database server's port number
```

```
Enter database super user password [ ] :mypassword  
##Enter the superuser password
```

```
Please enter CIDR formatted network address range  
that agents will connect to the server from, to be  
added to the server's pg_hba.conf file. For  
example, 192.168.1.0/24 [ 0.0.0.0/0 ] :  
##Keep default
```

```
Enter database systemd unit file or init script  
name (i.e. edb-as-12 or postgresql-12, etc.) [ ]  
:edb-as-17  
## Enter database service name
```

```
Please specify agent certificate path (Script will  
attempt to create this directory, if it does not  
exists) [ /root/.pem/ ] : <HIT ENTER>  
##Keep default
```

```
Please specify PEM web server (openssl)
```

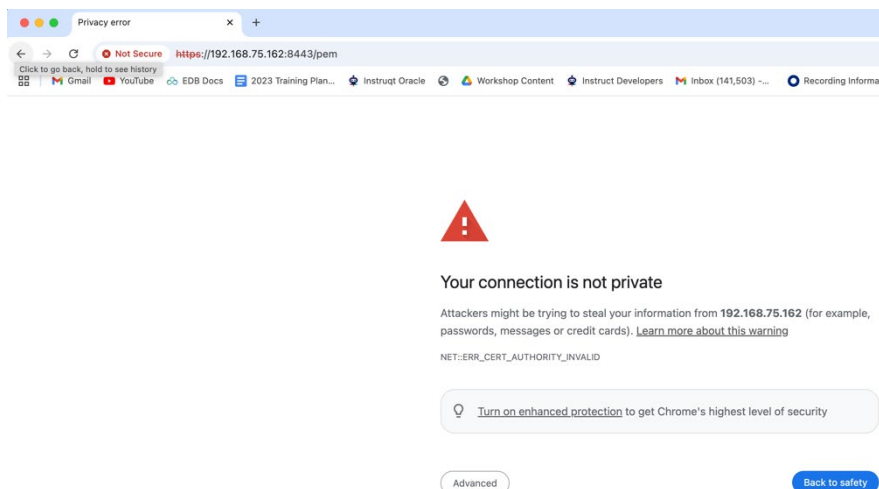
```
certificate subject, template:
/C=CountryCode/ST=StateName/L=CityName/O=CompanyNa
me/OU=CompanySectionName/CN=CommonNameOrHostname/e
mailAddress=EmailAddress [ ]
:/C=US/ST=MA/L=Bedford/O=EDB/OU=EDB Postgres
Manager - HTTPD
Server/CN=<hostname>/emailAddress=test@enterprised
b.com
##Provides the custom web server certificate
subject in given format
```

- c. Once the installation and configuration is done you can access the PEM Web Client using url : **<https://localhost:8443/pem>**

Note: You can specify the ip address of PEM Server while accessing PEM Web Client from web browser on remote machine. Remember the port 8443 must be open in firewall.

2. Access the Postgres Enterprise Manager

- a. Connect using the PEM Web Client. Go to web browser and write the url : <https://localhost:8443/pem>



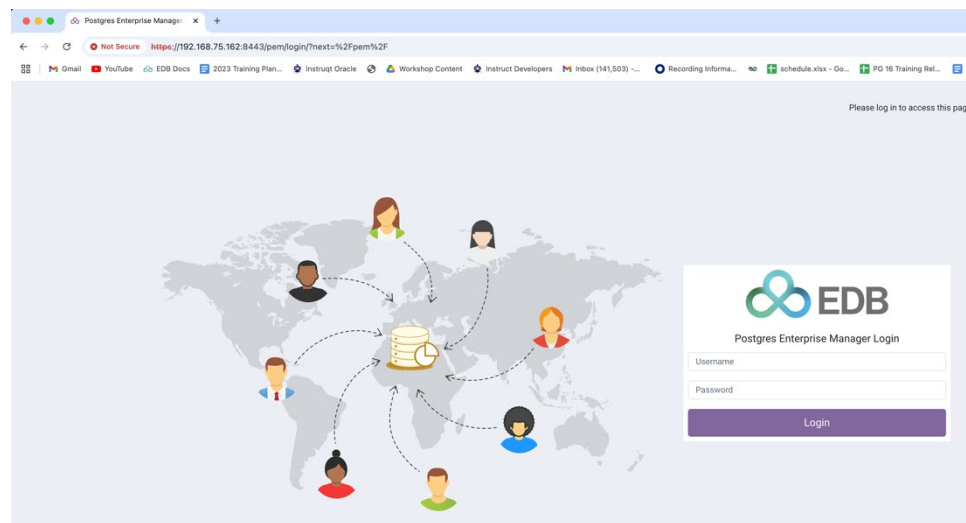
You may have to add certificate exception in your browser window.

- b. To access PEM remotely, connect to PEM using PEM server IP address.

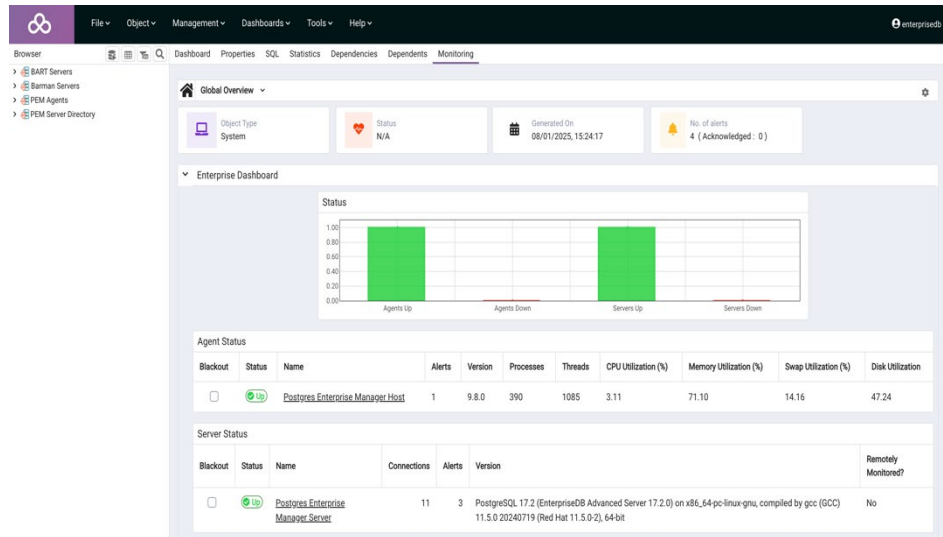
E.g: `https://3.98.120.119:8443/pem`

Please ensure port 80 is open on the network firewall for remote access.

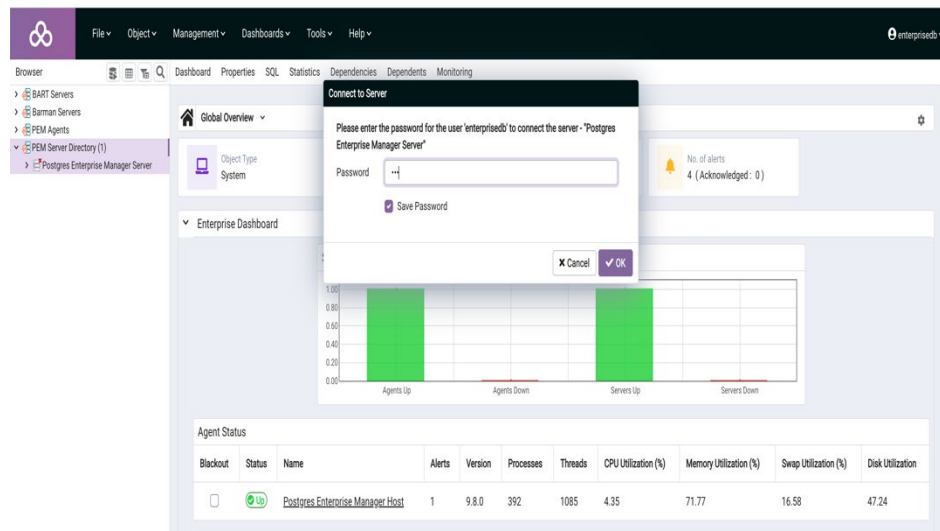
- c. Login to the PEM. You can use the default `enterprisedb` user and password.



3. Open PEM Client and connect to Postgres Enterprise Manger Server:



d. Click on Postgres Enterprise Manager and provide password for enterisedb user



SQL Tuning

Lab Exercise 1

1. You are working as a DBA. Users are complaining about long running queries and high execution times. Configure your database instance to log slow queries. Any query taking more than 5 seconds must be logged.
2. After logging slow queries you found following query taking longer than expected time

```
SELECT * FROM customers JOIN orders USING(customerid);
```
3. View the explain plan of the above query.
4. View the execution time for the above query in `edb-psql` terminal.

Answer Lab Exercise – 1

1. Open a Terminal and login as **enterprisedb**. Type

```
su - enterprisedb
```
2. Edit the **postgresql.conf** file to log slow running queries. Type

```
vi /var/lib/edb/as17/data/postgresql.conf
```
3. Change the parameter `log_min_duration_statement` to 5000
4. Save and close the file Type `<ESC>:wq <Enter>`
5. Reload the server. Type

```
pg_ctl -D /var/lib/edb/as17/data reload
```
6. Go to the `edbstore` database as the **edbuser** Type

```
psql edbstore edbuser
```

 and enter the edbuser password
7. View the explain plan. Type

```
EXPLAIN SELECT * FROM customers JOIN orders  
USING(customerid);
```
8. View the execution time of the above query. Type

```
EXPLAIN ANALYZE SELECT * FROM customers JOIN orders  
USING(customerid);
```

Lab Exercise - 2

1. Create a table using following queries:

```
- CREATE TABLE lab_test1 (c1 int4, c2 int4);  
- INSERT INTO lab_test1(c1, c2)  
  values(generate_series(1, 100000), 1);  
- INSERT INTO lab_test1(c1, c2)  
  values(generate_series(100001, 200000),  
        2);  
- INSERT INTO lab_test1(c1, c2)  
  values(generate_series(200001, 300000),  
        3);
```

2. Create three partial indexes on lab_test1 table as following:

Index Name	Predicate
idx1_c1	c1 between 1 and 100000
idx2_c1	c1 between 100001 and 200000
idx3_c1	c1 between 200001 and 300000

Answer Lab Exercise – 2

1. Open a Terminal. Login as **enterprisedb** and connect to edbstore database using **edbuser**
2. Create a table and insert data. Type

```
CREATE TABLE lab_test1 (c1 int4, c2 int4);  
INSERT INTO lab_test1(c1, c2) values(generate_series(1,  
100000), 1);  
INSERT INTO lab_test1(c1, c2)  
values(generate_series(100001, 200000), 2);  
INSERT INTO lab_test1(c1, c2)  
values(generate_series(200001, 300000), 3);
```
3. Create the indexes. Type

```
CREATE INDEX idx1_c1 ON lab_test1 (c1) WHERE c1 between 1  
and 100000 ;  
CREATE INDEX idx2_c1 ON lab_test1 (c1) WHERE c1 between  
100001 and 200000 ;  
CREATE INDEX idx3_c1 ON lab_test1 (c1) WHERE c1 between  
200001 and 300000 ;
```
4. Exit psql type \q

Lab Exercise – 3

1. Detect the index usage for all the user indexes in `edbstore` database.
2. Reindex all the indexes
3. Manually update the statistics for all the objects in `edbstore` database

Answer Lab Exercise – 3

1. Connect to the `edbstore` database. Type

```
psql edbstore edbuser
```
2. Detect the index usage for all the user indexes in `edbstore` database. Type

```
SELECT * FROM pg_stat_user_indexes;
```

```
edbstore=> select * from pg_stat_user_indexes;
```

relid	indexrelid	schemaname	relname	indexrelname	idx_scan	idx_tup_read	idx_tup_fetch
24840	24908	edbuser	categories	categories_pkey	0	0	0
24848	24910	edbuser	customers	customers_pkey	0	0	0
24856	24912	edbuser	dept	dept_dname_uq	0	0	0
24856	24914	edbuser	dept	dept_pk	0	0	0
24859	24916	edbuser	emp	emp_pk	0	0	0
24863	24918	edbuser	inventory	inventory_pkey	0	0	0
24872	24920	edbuser	jobhist	jobhist_pk	0	0	0
24887	24922	edbuser	orders	orders_pkey	0	0	0
24892	24924	edbuser	products	products_pkey	0	0	0
24845	24926	edbuser	cust_hist	ix_cust_hist_customerid	0	0	0
24848	24927	edbuser	customers	ix_cust_username	0	0	0
24887	24928	edbuser	orders	ix_order_custid	0	0	0
24884	24929	edbuser	orderlines	ix_orderlines_orderid	0	0	0
24892	24930	edbuser	products	ix_prod_category	0	0	0
24892	24931	edbuser	products	ix_prod_special	0	0	0

(15 rows)

3. Reindex all the indexes. Type `REINDEX Database edbstore;`

```
edbstore=# REINDEX database edbstore;
REINDEX
```

4. Manually update the statistics for all the objects in `edbstore` database. Type

```
ANALYZE;
```

```
edbstore=# ANALYZE;
ANALYZE
```

5. Exit `psql` type `\q`

Performance Tuning

Lab Exercise – 1

1. Users are complaining about slower than normal performance on `edbstore` database
 - Tune **postgresql.conf** parameters for optimal performance based on the `edbstore` database size, largest table, and other necessary information collected from `edbstore` database
 - Change maximum concurrent connections on the cluster to 50

Answer Lab Exercise – 1

1. Open server parameter file and change the parameters as per your requirements. You may need to uncomment some parameters. Open server parameter file using:

```
vi /var/lib/edb/as17/data/postgresql.conf
```
2. Uncomment and Change following:
 - a. `max_connections = 50`
 - b. Save and close the file. Type `<ESC>:wq <Enter>`
3. Restart your cluster. Login as root or sudo user. Type

```
systemctl restart edb-as-17
```

Lab Exercise – 2

1. Write a statement to load `customers` table from `edbstore` database to the PostgreSQL buffer cache

Answer Lab Exercise – 2

1. Go to the `edbstore` database as **enterprisedb**. Type

```
psql edbstore enterprisedb
```
2. Create the **pg_prewarm** extension on the `edbstore` database. Type

```
CREATE EXTENSION pg_prewarm;
```
3. Execute **pg_prewarm** on the `customers` table. Type

```
SELECT pg_prewarm ('edbuser.customers');
```
4. Exit out of `edbstore`. Type

```
\q
```

Lab Exercise – 3

1. `pg_test_fsync` can determine the fastest `wal_sync_method` on a specific system. Run this tool on your local machine and determine the best `wal_sync_method` settings for your system.

Answer Lab Exercise – 3

1. Run the `pg_test_fsync` tool. Type `pg_test_fsync`

```
[enterprisedb@edblab ~]$ pg_test_fsync
5 seconds per test
O_DIRECT supported on this platform for open_datasync and open_sync.

Compare file sync methods using one 8kB write:
(in wal_sync_method preference order, except fdatasync is Linux's default)
    open_datasync          1438.028 ops/sec      695 usecs/op
    fdatasync              1432.135 ops/sec      698 usecs/op
    fsync                  729.286 ops/sec     1371 usecs/op
    fsync_writethrough      n/a
    open_sync              739.170 ops/sec     1353 usecs/op

Compare file sync methods using two 8kB writes:
(in wal_sync_method preference order, except fdatasync is Linux's default)
    open_datasync          791.913 ops/sec     1263 usecs/op
    fdatasync              1377.076 ops/sec      726 usecs/op
    fsync                  672.739 ops/sec     1486 usecs/op
    fsync_writethrough      n/a
    open_sync              409.701 ops/sec     2441 usecs/op

Compare open_sync with different write sizes:
(This is designed to compare the cost of writing 16kB in different write
open_sync sizes.)
    1 * 16kB open_sync write      716.588 ops/sec     1396 usecs/op
    2 *  8kB open_sync writes     342.580 ops/sec     2919 usecs/op
    4 *  4kB open_sync writes     165.697 ops/sec     6035 usecs/op
    8 *  2kB open_sync writes      83.685 ops/sec    11950 usecs/op
   16 *  1kB open_sync writes     41.317 ops/sec    24203 usecs/op

Test if fsync on non-write file descriptor is honored:
(If the times are similar, fsync() can sync data written on a different
descriptor.)
    write, fsync, close          671.084 ops/sec     1490 usecs/op
    write, close, fsync          695.124 ops/sec     1439 usecs/op

Non-sync'ed 8kB writes:
    write                        654364.113 ops/sec        2 usecs/op
[enterprisedb@edblab ~]$
```

Based on the above result of `pg_test_fsync`, `open_datasync` and `fdatasync` are the best choices for server parameter `wal_sync_method`. The default for `wal_sync_method` on Linux is `fdatasync`

Extensions

Lab Exercise – 1

In this lab, you will be working with a Postgres extension to enhance the functionality of your database.

1. Install and configure extension named **hstore** in **edbstore** database
2. Perform a basic test or use case that demonstrates the functionality provided by the extension

Answer Lab Exercise - 1

1. Go to the edbstore database as postgres. Type

```
psql edbstore enterprisedb
```

2. Install the extension hstore. Type

```
CREATE EXTENSION IF NOT EXISTS hstore;
```

3. Here is a sample use case for hstore extension.

```
--Create a table with hstore column:
```

```
CREATE TABLE messages (  
    message_id SERIAL PRIMARY KEY,  
    sender_id INT,  
    receiver_id INT,  
    message_text TEXT,  
    metadata HSTORE  
);
```

```
--Multirow Insert statement to add six rows:
```

```
INSERT INTO messages (sender_id, receiver_id, message_text,  
metadata)  
VALUES
```

```
    (1, 2, 'Hello, how are you?', 'sent_at => "2023-01-07  
12:30:00", priority => "low"'),
```

```
    (2, 1, 'All is well. How about you?', 'sent_at => "2023-  
01-07 12:31:00", priority => "low"'),
```

```
    (1, 2, 'Fine Thank You. What is the release data for  
PostgreSQL 16', 'sent_at => "2023-01-07 12:32:00", priority =>  
"high"'),
```

```
    (2, 1, 'Its planned for Oct 2023', 'sent_at => "2023-01-07  
12:35:00", priority => "high"'),
```

```
    (1, 3, 'Oct 2023', 'sent_at => "2023-01-07 12:37:00",  
priority => "high", type => "Marketing"'),
```

```
    (1, 4, 'Oct 2023', 'sent_at => "2023-01-07 12:38:00",  
priority => "high", type => "Sales");
```

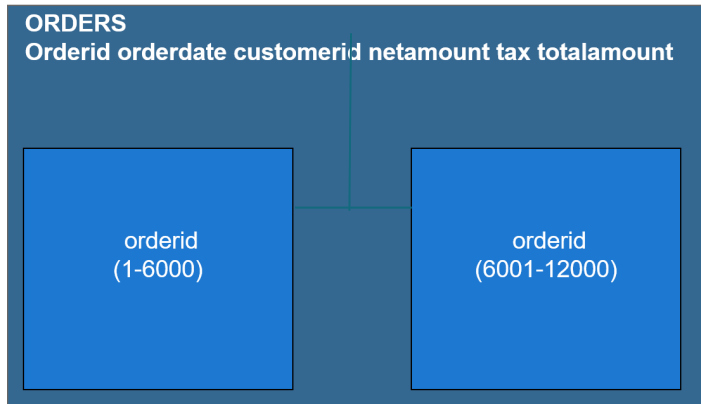
```
--List high priority messages:
```

```
SELECT message_id, sender_id, receiver_id, message_text FROM messages  
where metadata -> 'priority' = 'high';
```

Table Partitioning

Lab Exercise – 1

1. View the structure of `orders` table in `edbstore` database
2. Partition the `orders` table as shown in following diagram:



Answer Lab Exercise – 1

1. Open a terminal and login as **enterprisedb**. Type `su - enterprisedb` and type the `enterprisedb` password.
2. Connect to `edbstore` database using **edbuser** user. Type `psql edbstore edbuser`
3.

```
CREATE TABLE orders1 (LIKE orders)
PARTITION BY RANGE (orderid)
(PARTITION ord_1 VALUES LESS THAN ('6001'),
 PARTITION ord_2 VALUES LESS THAN ('12001'));
```
4. Load data into the `orders1` table. Type

```
INSERT INTO orders1 SELECT * FROM orders;
```
5. Create indexes on partitioned tables. Type

```
CREATE INDEX idx_orders1_1 ON orders1_ord_1 (orderid);
CREATE INDEX idx_orders1_2 ON orders1_ord_2 (orderid);
```
6. Review the data loaded. Type

```
SELECT COUNT(*) from orders1;
SELECT COUNT(*) FROM orders1_ord_1;
SELECT COUNT(*) FROM orders1_ord_2;
```

Lab Exercise - 2

1. Create primary key on `orderid` column of each partitioned table
2. View the execution plan of following query

```
SELECT * FROM orders1 WHERE orderid=3000;
```
3. Turn off the **constraint_exclusion** and view execution plan of following query

```
SELECT * FROM orders1 WHERE orderid=3000;
```

Answer Lab Exercise – 2

1. Create a primary key for partitions of orders1. Type

```
ALTER TABLE orders1 ADD primary key(orderid);
```
2. Run and Explain plan on the orders1 table. Type

```
EXPLAIN SELECT * FROM orders1 WHERE orderid=3000;
```
3. Turn off **constraint_exclusion** and view plan. Type

```
SET constraint_exclusion=off;
```
4. Run the explain plan on orders1 again. Type

```
EXPLAIN SELECT * FROM orders1 WHERE orderid=3000;
```